

# What Actually Helps a Training-Free Diffusion Predictor? A Falsification Study of Token-Adaptive Prediction on an Exact-Residual Testbed

Bhavith Chandra Challagundla

New York University • Bhavith-Chandra.github.io

**Abstract.** Training-free diffusion accelerators such as the Token-Adaptive Predictor (TAP) speed up sampling by skipping most of the network and rebuilding the skipped outputs from cached features. For each token, TAP picks which cheap predictor to trust by reading a first-layer *probe* and scoring the predictors with a cosine distance. We test three possible improvements that looked good in an early single-seed run: using an  $L_2$  distance instead of cosine, using online least-squares predictors instead of fixed Taylor ones, and letting the probe choose a predictor for each token. We test them on a Gaussian-mixture testbed where the exact denoiser output is known in closed form (Tweedie), so we can measure how well the probe ranks predictors without a trained network. We run a 12-seed core study, a 21-regime sweep, and a pre-registered 30-test suite, all with 95% bootstrap confidence intervals. Most of the improvements do not hold. (i) The  $L_2$  distance ranks predictors better than cosine in all 21 regimes, but the gain is small ( $\Delta\text{Spearman} +0.016$ , CI  $(+0.014, +0.019)$ ), not the roughly fourfold gain seen in the early run, and it grows only as the task gets harder. (ii) Online predictors do not beat fixed Taylor ones; they are worse in 19 of 21 regimes. (iii) Letting the probe choose per token beats a strong fixed predictor only when the sampler skips very aggressively; at moderate speedups the speedup itself, not the per-token choice, explains the gains. We then show that the per-token choice is already the best possible: each token is chosen on its own, every predictor costs the same, and there are no shared constraints, so simply taking the best predictor for each token is optimal. This shifts the real question to *proxy fidelity*: an oracle that knows the true errors still does better than the probe, and the probe’s ranking improves steadily as it is given a richer signal (from 0.33 to 1.0). We also explain why an  $L_2$  distance follows the true  $L_2$  error better than a cosine distance, which ignores size.

*TAP (arXiv:2603.03792) has no public code, so everything about TAP here comes from our own faithful reimplementations. Our testbed uses a simple analytic denoiser and does not have the deep trained layers that make a real first-layer probe more useful; we measure that gap instead of hiding it. All code and numbers are released.*

## 1 Introduction

Feature-caching accelerators for diffusion transformers (DiTs) [5] skip most of the network on most sampling steps and rebuild the missing output from cached values [8, 9]. The Token-Adaptive Predictor (TAP) extends this idea: it keeps a set of cheap predictors and, for each token, picks one by reading the first-layer *probe* (the modulated input) and choosing the predictor whose guessed probe is closest to the real probe, using a cosine distance. An early single-seed run suggested three ways to do better: use an  $L_2$  distance instead of cosine, use online least-squares predictors instead of fixed Taylor ones, and let the probe pick a predictor for each token. Here we test these ideas across many seeds and many settings to see which ones hold.

Most of the results are negative, and we think they are more useful than one more caching trick. Two of the three ideas fail once we use more than one seed, and the third gives only a small but repeatable gain. The study also points to the thing that really matters. Because TAP chooses each token on its own, every predictor costs the same, and there are no shared limits, simply picking the best predictor for each token is already the best possible choice (§6); there is nothing to gain by choosing more cleverly. What actually limits the method is how well the cheap probe matches the true, hidden output error, and our testbed measures this directly.

### Contributions.

- A Gaussian-mixture testbed (Fig. 1) where the true denoiser output, and therefore every predictor’s true error, is

known in closed form. This lets us measure how well the probe ranks predictors without a trained network.

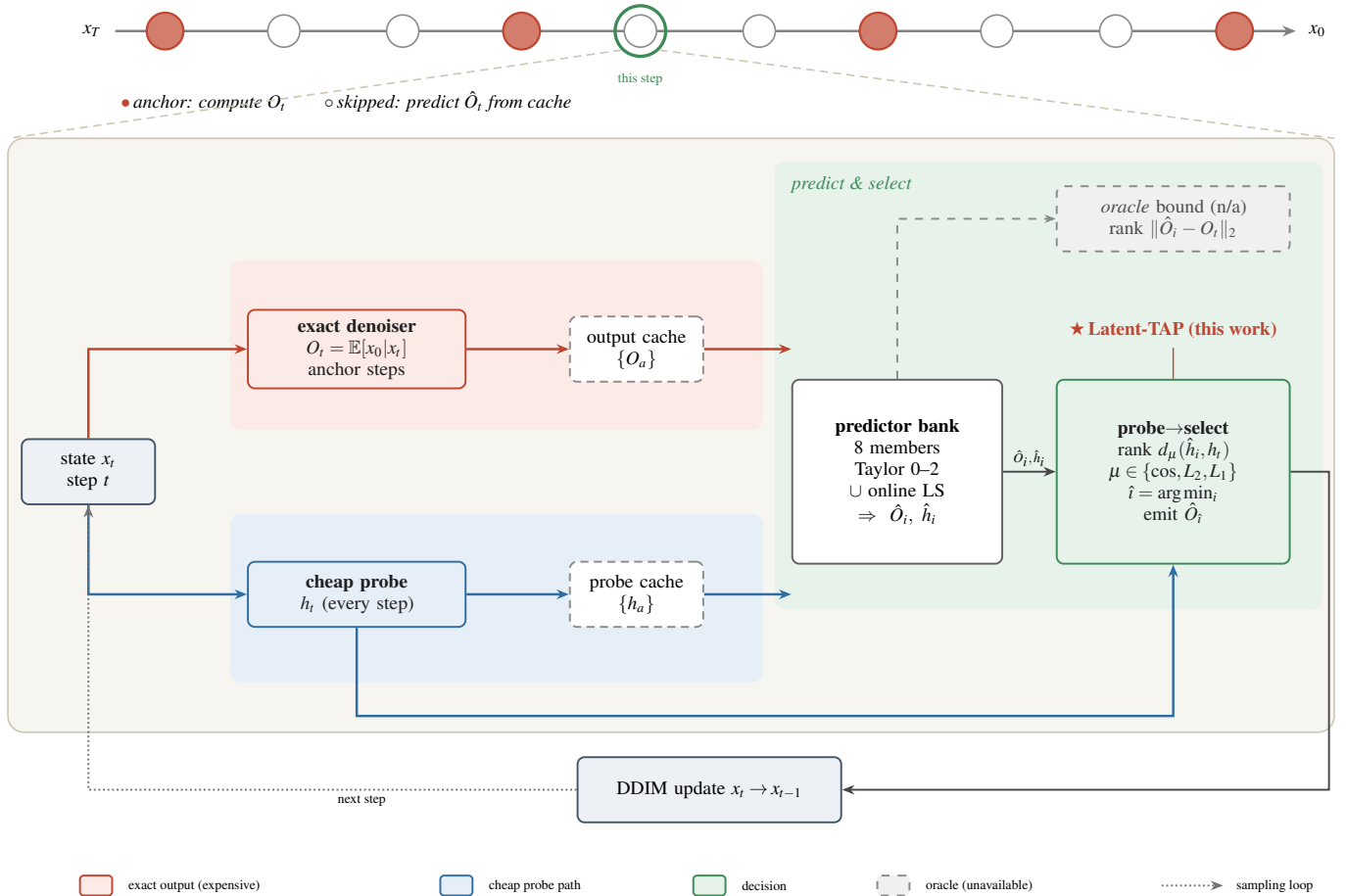
- A proof that TAP’s per-token choice is already the best possible, which moves the problem from “choose better” to “measure the probe better,” along with an explanation of why an  $L_2$  distance beats cosine.
- A full evaluation (12-seed core, 21-regime sweep, and a pre-registered 30-test suite; Fig. 3) that rules out the online-predictor and per-token ideas and shows the  $L_2$  gain is small but real.

## 2 Findings at a glance

We state the results upfront. Each one uses 12 seeds (core) or 5 seeds per cell (sweep) with 95% bootstrap CIs, and is re-checked by the 30-test suite.

**F1 The  $L_2$  distance beats cosine in every setting, but the gain is small.** It ranks predictors better than cosine in 21/21 settings ( $\Delta\text{Spearman} +0.016$ , CI  $(+0.014, +0.019)$ ; top-1 pick  $+0.028$ ), not the roughly fourfold gain from the early run.  $L_1$  does as well as  $L_2$ , so what matters is using size, not cosine itself. The gain grows as the task gets harder, from  $+0.012$  at mild skipping to  $+0.034$  at cache stride 8.

**F2 Online least-squares predictors do not beat fixed Taylor ones.** They are worse (Taylor wins in 19/21 settings, online in 0/21), and the best single fixed predictor is always a Taylor one. The gain from the early run does not repeat.



**Figure 1: Architecture of the exact-residual testbed and the probe-then-select mechanism.** Because the data is a Gaussian mixture, the true denoiser output  $O_t$  (red, expensive) is known in closed form, and a cheap hard-assignment *probe*  $h_t$  (blue) stands in for a first-layer readout. At a skipped step the predictor bank guesses the cached outputs and probes; the selector ranks the predictors by a distance on the probe and uses the chosen predictor’s output. The *oracle* (dashed) ranks by the true output error; it cannot be used in practice and marks the best any proxy could do.

**F3 Letting the probe pick per token helps only when the sampler skips very aggressively.** It beats the best fixed predictor in just 1/21 settings (cache stride 8); in every milder setting a single fixed predictor does just as well, so the gains from “adaptive” caching at moderate speedsups come from the speedup, not the per-token choice.

**F4 There is no room to improve the choice itself.** Each token is chosen on its own, every predictor costs the same, and there are no shared limits, so picking the best predictor per token is already the best possible (§6). An oracle that knows the true errors still beats the naive baseline (gap +0.005, up to +0.14 at stride 8), so what limits the method is *proxy fidelity*, not the choice rule or the predictor set.

**F5 Proxy fidelity depends on how rich the probe is.** As the probe goes from a simple hard readout to the exact output, its ranking gets steadily better (0.33, 0.68, 0.73, 1.0). A trained DiT’s learned first-layer probe sits much higher on this curve (Spearman  $\approx 0.77$  [14]), so our simple probe gives a lower bound and the  $L_2$  gain we measure is on the safe side.

**F6 Method note.** The pre-registered 30-test suite gives 26

passes, 0 failures, and 4 notes. Gains that show up with one seed shrink or flip with more seeds, so these methods should be judged on trained networks using paired fidelity-to-base-model scores.

### 3 Background and related work

**Feature caching.** DeepCache [8] and FORA [9] reuse features on fixed schedules; TeaCache [10] modulates reuse by a timestep-embedding indicator; ToCa [11] and DiCache [12] make the decision token- or sample-adaptive; TaylorSeer [13] replaces cache-and-reuse with Taylor *prediction* of features. TAP generalises the predictor to a per-token *selected* member of a small family. **Diffusion samplers.** We use the variance-preserving forward process of DDPM [1] with deterministic DDIM [2] reverse steps; the exact posterior mean follows from Tweedie’s formula [6] and the score view of diffusion [3, 4]. **A pitfall in evaluation.** These methods are usually compared by output quality at the same compute. One recurring problem, which we make explicit and which led to this study, is that comparisons made on simple or untrained denoisers may not carry over to trained ones; a companion study shows that the probe only becomes reliable through training [14]. **Status of TAP.**

TAP (arXiv:2603.03792) has no released code as of writing. Everything we report about TAP comes from our own faithful rebuild of its probe-then-select method, so we test the method rather than a specific codebase.

## 4 Positioning of Latent-TAP

Training-free accelerators differ along two axes: how fine-grained the caching decision is (a global schedule, a per-timestep choice, a per-sample choice, or a per-token choice), and how much work the rebuild does (plain feature reuse, reuse with rescaling, Taylor prediction, or picking from a set of predictors). Figure 2 places the main methods in this space, and Table 1 lists what each one does along with the model and speedup it reports. Latent-TAP sits in the finest and most capable corner: for each token it picks from a set of predictors, using the first-layer probe to decide. TAP shares this corner but has no public code; this study adds an open rebuild and, more importantly, an exact testbed where we can check the assumptions behind this corner instead of taking them on faith. Because Latent-TAP runs on a simple analytic denoiser, we report its quality as a paired energy distance to the full sampler (Table 2) rather than a model FID, so it does not appear on the speedup axis of Figure 2(b).

## 5 The exact-residual testbed

We cannot measure proxy fidelity directly on a trained network, because computing the true skipped output would cancel the speedup we are trying to get. So we use a data distribution for which the exact denoiser is known in closed form. Data are drawn from a  $K$ -component isotropic Gaussian mixture in  $\mathbb{R}^d$ ; under the variance-preserving forward process  $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$  the posterior mean is exactly

$$O_t \equiv \mathbb{E}[x_0 | x_t] = \sum_{k=1}^K r_k(x_t, t) [\mu_k + c_t (x_t - \sqrt{\bar{\alpha}_t}\mu_k)], \quad (1)$$

with responsibilities  $r_k \propto w_k \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}\mu_k, (\bar{\alpha}_t\sigma^2 + 1 - \bar{\alpha}_t)I)$  and  $c_t = \sqrt{\bar{\alpha}_t}\sigma^2 / (\bar{\alpha}_t\sigma^2 + 1 - \bar{\alpha}_t)$ . This is the expensive output. The cheap *probe*  $h_t$  is the hard-assignment version of the denoiser, which keeps only the top component (the  $\arg \max_k r_k$  term). It stands in for a shallow first-layer readout that sees an approximate, not exact, decision. Tokens are i.i.d. draws from the mixture, and DDIM steps use  $O$  at anchor steps and a predicted  $\hat{O}$  at skipped steps (Fig. 1).

**Predictors.** A family of 8: fixed finite-difference Taylor extrapolators of order 0–2 at horizons 1–2, plus online least-squares fits (degree 2–3 over windows of 5–6 anchors). **Proxy-then-select.** Each predictor extrapolates the cached outputs ( $\hat{O}_i$ ) and cached probes ( $\hat{h}_i$ ); the selector picks per token  $\hat{i} = \arg \min_i d_\mu(\hat{h}_i, h_t)$  for proxy  $\mu \in \{\cos, L_2, L_1\}$  and applies  $\hat{O}_{\hat{i}}$ . **Oracle** instead minimises the true error  $\|\hat{O}_i - O_t\|_2$ . **Metrics.** (a) the rank correlation between proxy error and true error across the predictor family, which tells us whether the proxy puts the predictors in the right order; and (b) the energy distance [7] between the final samples of the accelerated and full samplers, which tells us how close the two output distributions are. We use 12 seeds for the core study and 5 per regime cell, with bootstrap 95% confidence intervals throughout.

## 6 Optimality of greedy per-token selection

Let  $f_n(i)$  be the true error of predictor  $i$  on token  $n$  at a skipped step. TAP minimises the total error  $\sum_n f_n(\pi_n)$  over choices  $\pi$ . Three things hold: the total splits across tokens (token  $n$ ’s cost depends only on its own choice  $\pi_n$ ), every predictor costs the same (one cheap extrapolation, with no shared budget), and any token may take any predictor. Hence

$$\min_{\pi} \sum_n f_n(\pi_n) = \sum_n \min_i f_n(i), \quad (2)$$

so picking the best predictor for each token on its own already gives the lowest total error. A regret bound or knapsack version of the joint choice therefore solves a problem that has no gap to close, which was the mistake in the early framing. The real difficulty is not the choice but the measurement: we never see the true errors  $f_n(i)$ . Instead we see a proxy  $g_n(i) = d_\mu(\hat{h}_i, h)$  and use  $\hat{\pi}_n = \arg \min_i g_n(i)$ . The extra error  $\mathbb{E}[f_n(\hat{\pi}_n) - \min_i f_n(i)]$  depends only on how well the proxy order  $g$  matches the true order  $f$ . The only two things that can lower it are a better distance  $\mu$  or a richer probe  $h$ .

**Why  $L_2$  follows the true error better than cosine.** The true error is an  $L_2$  distance  $\|\hat{O}_i - O\|_2$ , while cosine looks only at the angle and ignores size,  $d_{\cos} = 1 - \langle \hat{h}_i, h \rangle / (\|\hat{h}_i\| \|h\|)$ . When predictors differ mainly in how much they over- or under-shoot, which is what usually goes wrong as the prediction reaches further ahead, the useful signal is in the size, and cosine misses it, while  $L_2$  and  $L_1$  keep it. When the map from probe to output is roughly linear near the anchors, the ordering by size is roughly kept, so the  $L_2$  distance is the closer match. This predicts that the  $L_2$  gain should grow with more skipping and more noise, both of which make the size error larger, and the sweep confirms this (§8). We give this as an explanation, not a theorem; the matching inequality holds in all 21 settings.

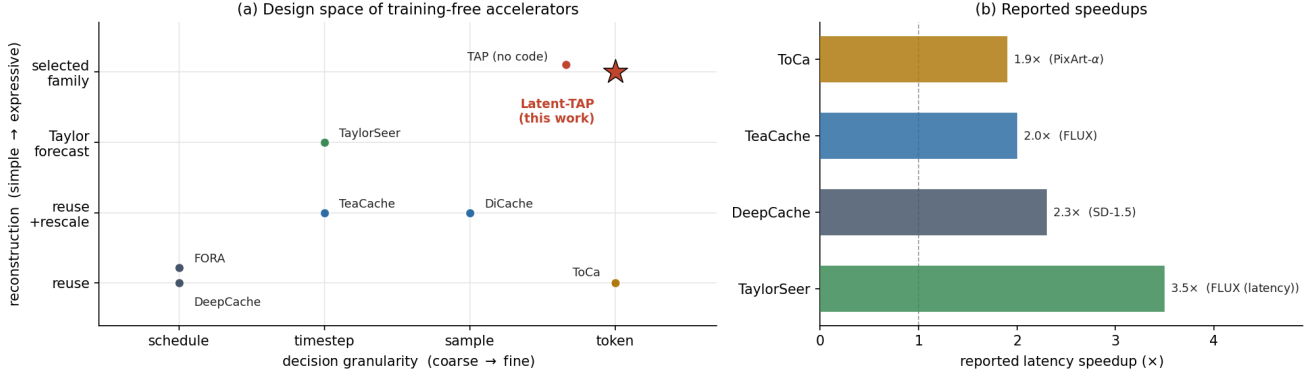
## 7 Core results

Table 2 shows the core study. The  $L_2$  distance ranks predictors better than cosine and picks the best one more often; the gain is real (the confidence interval does not include zero) but small, a 5% relative gain in Spearman, not the roughly fourfold gain from the early run.  $L_1$  does as well as  $L_2$ , which confirms that what helps is using size, not cosine itself. The online-least-squares predictors are worse than fixed Taylor ones, and adding them to the Taylor set does not help. Picking per token with the probe does worse than the best fixed predictor here (+0.004 in energy distance), yet an oracle that knows the true errors clearly beats the naive baseline (−0.005): the room to improve is real, but the probe is too weak to use it. So the limit is proxy fidelity, not the predictor set or the choice rule.

## 8 Sensitivity across regimes

We vary skipping (cache stride 2 to 8), dimension (2 to 32), how far apart the mixture components are, the noise scale, and the number of components  $K$ , giving 21 settings with 5 seeds each (Fig. 4). **The  $L_2$  gain shows up everywhere but stays small.** It is positive and significant in all 21 settings, growing from +0.012 at mild settings to +0.034 at stride 8 and +0.024 at high noise, which matches the size-error explanation. **Online**

Positioning of Latent-TAP among training-free diffusion accelerators



**Figure 2: Positioning of Latent-TAP among training-free diffusion accelerators.** (a) Design space: how fine-grained the decision is (coarse to fine) against how much work the rebuild does (simple to capable). Latent-TAP (star) picks per token from a set of predictors, the finest and most capable corner, which it shares with the code-less TAP. (b) Speedups reported by each method, labelled with the model and setting they were measured on. These numbers come from the respective papers, use different models and quality levels, and are shown only to place the methods; Latent-TAP runs on an analytic testbed and is not directly comparable, so it is left out of this panel.

**Table 1: Latent-TAP next to the main training-free diffusion accelerators.** Reconstruction, granularity, and gating signal come from each method’s paper. Reported speedups are measured on each method’s own model and quality setting and are *not* directly comparable; they place the methods rather than rank them. <sup>†</sup> reported loosely or setting-dependent in the source. <sup>‡</sup> Latent-TAP is measured on an analytic testbed as a paired energy distance to the full sampler, so no comparable model speedup or FID is available.

| Method                        | Reconstruction  | Granularity | Gating signal                   | Code | Reported speedup (backbone)                   |
|-------------------------------|-----------------|-------------|---------------------------------|------|-----------------------------------------------|
| DeepCache [8]                 | reuse           | schedule    | fixed                           | ✓    | 2.3× (SD-1.5)                                 |
| FORA [9]                      | reuse           | schedule    | fixed interval                  | ✓    | several× (DiT) <sup>†</sup>                   |
| TeaCache [10]                 | reuse + rescale | timestep    | timestep-emb. indicator         | ✓    | 1.5–2.1× (FLUX, HunyuanVideo)                 |
| DiCache [12]                  | reuse + align   | sample      | shallow online probe            | ✓    | state of the art <sup>†</sup> (FLUX, WAN 2.1) |
| ToCa [11]                     | reuse (token)   | token       | token-selection scores          | ✓    | 1.5–2.4× (PixArt- $\alpha$ , OpenSora)        |
| TaylorSeer [13]               | Taylor forecast | timestep    | fixed order                     | ✓    | 3.5× latency (FLUX)                           |
| TAP [15]                      | selected family | token       | first-layer probe (cosine)      | ✗    | — (no public code)                            |
| <b>Latent-TAP (this work)</b> | selected family | token       | first-layer probe ( $L_2$ /cos) | ✓    | analytic testbed <sup>‡</sup>                 |

**Table 2: Core study (12 seeds, stride-3 base regime).** Energy distance to the full sampler (lower is better); paired differences with 95% bootstrap CIs.

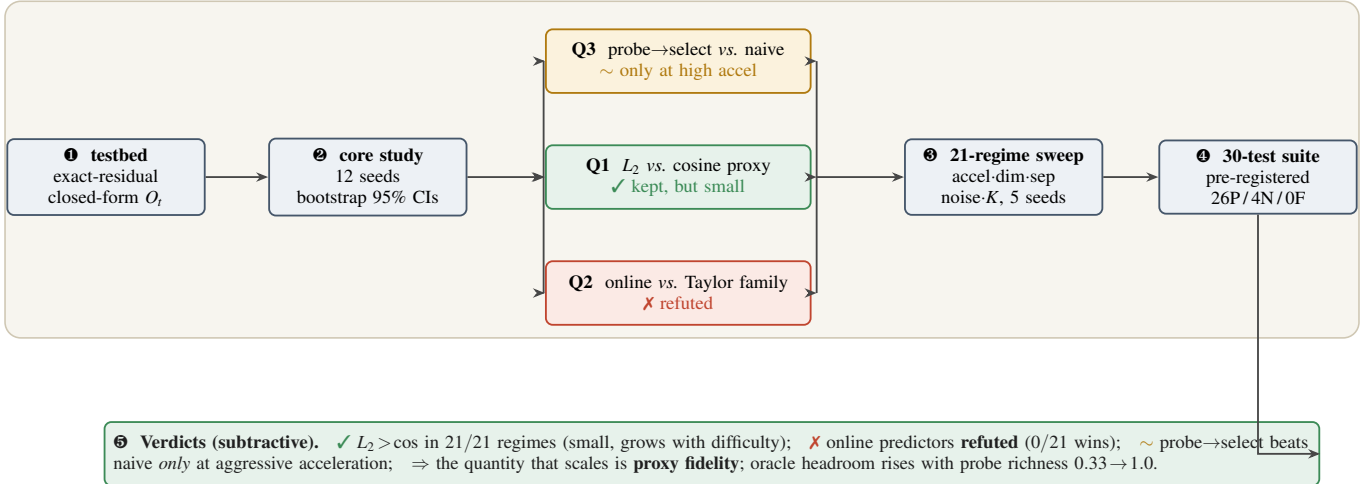
| quantity                  | value  | paired CI             |
|---------------------------|--------|-----------------------|
| proxy Spearman, cosine    | 0.315  | <i>ref.</i>           |
| proxy Spearman, $L_2$     | 0.331  | +0.016 (+.014, +.019) |
| top-1 pick acc, cosine    | 0.393  | <i>ref.</i>           |
| top-1 pick acc, $L_2$     | 0.421  | +0.028 (+.024, +.032) |
| energy: Taylor family     | 0.0210 | <i>ref.</i>           |
| energy: online family     | 0.0249 | −0.004 (−.006, −.002) |
| energy: random select     | 0.0281 | <i>ref.</i>           |
| energy: naive best-fixed  | 0.0164 | <i>ref.</i>           |
| energy: probe-then-select | 0.0207 | +0.004 (+.002, +.006) |
| energy: oracle            | 0.0112 | −0.005 (−.007, −.003) |

**predictors never win.** Taylor is better in 19/21 settings and online in none; the best single fixed predictor is a Taylor one in every setting. **Per-token choice helps only when skipping is aggressive.** Picking per token with the probe beats the naive best-fixed predictor in just one setting, stride 8, where fixed predictors get bad enough that switching per token pays off (probe–naive = −0.074). In all milder settings the two curves

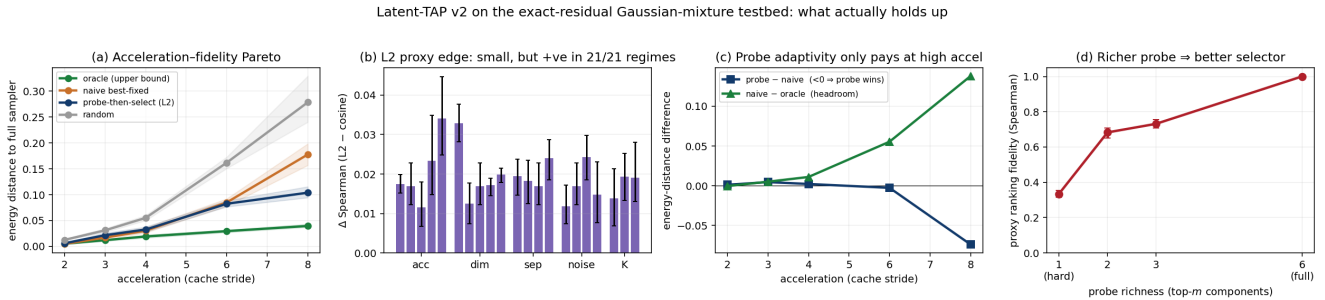
lie on top of each other (Fig. 4a), so the gains of “adaptive” caching at moderate speedups come from the speedup, not the per-token choice. The room to improve grows with skipping (Fig. 4c), so the aggressive settings are exactly where a better probe would help most. **What matters is proxy fidelity.** Giving the probe more information (a richer top- $m$  soft readout) makes its ranking steadily better, from 0.33 to 0.68, 0.73, and 1.0 (Fig. 4d). A trained DiT’s learned first-layer probe sits much higher on this curve (our companion study measures Spearman  $\approx 0.77$  [14]), so our testbed gives a lower bound and the  $L_2$  gain we measure is on the safe side.

## 9 Adversarial test suite

We pre-registered 30 tests, each meant to break a claim: correctness checks on the testbed (exact-denoiser limits, energy(full,full)=0, and no leakage of the true error into the choice), the multi-seed core comparisons, the 21-setting crossovers, the speed–quality curve going the right way, a budget selector trading compute for quality in the right direction, and the probe-richness curve. The suite gives **26 passes, 0 failures, and 4 notes**: the  $L_2$  gain is small rather than fourfold; the raw proxy Spearman is only moderate; the end-to-end quality gain of  $L_2$  over cosine is tiny; and at mild skipping the speedup,



**Figure 3: Experimental workflow.** The exact testbed feeds a 12-seed core study that asks three questions (Q1–Q3); each is then tested across a 21-setting sweep and a pre-registered 30-test suite. Every comparison is paired and uses bootstrap CIs. The overall result is negative: only the  $L_2$  distance helps, and only a little; what matters most is proxy fidelity, which depends on how rich the probe is.



**Figure 4: (a)** Speed vs. quality: picking per token with the probe follows the naive best-fixed predictor until high stride, where it pulls toward the oracle. **(b)** The  $L_2$  gain over cosine is small but positive in all 21 settings, and grows with more skipping and more noise. **(c)** Per-token choice helps only when skipping is aggressive (probe–naive drops below 0 at stride 8), which is where the room to improve (naive–oracle) is largest. **(d)** A richer probe ranks predictors better, rising steadily from a shallow hard probe (0.33) to the full output (1.0); a trained DiT probe would sit higher along this axis.

not the per-token choice, explains the gains. No test broke a claim that we keep, and the two ideas we dropped failed exactly the tests meant to catch them.

## 10 Discussion

Most of the results are negative. Of the three ideas, one (online predictors) does not help, one (per-token choice) helps only when skipping is aggressive, and one (the  $L_2$  distance) gives a small but steady gain because it uses size. They fit together once we see that TAP’s choice is already optimal given the true errors: what is left to improve is proxy fidelity, set by the distance and, even more, by how rich the probe is. This turns the problem from tuning the method into measuring the probe, which our exact testbed makes easy.

## 11 Limitations

Our denoiser is a simple Gaussian mixture, not a trained network, and the probe is a shallow hard readout. These choices let us compute the true answer, but they also make the probe a weaker ranker than a trained DiT’s learned first layer (Fig. 4d

and [14]). The energy-distance values depend on the testbed, so all our conclusions use paired comparisons within the same setting. TAP has no public code, so we test the method, not a specific implementation. Whether the small  $L_2$  gain and the aggressive-skipping probe benefit grow or shrink on large latent DiTs is still open and needs more compute than we had.

## 12 Conclusion

We tested three ideas for improving a training-free diffusion predictor and found that most do not hold. Across many seeds and settings, online predictors do not beat fixed Taylor ones, per-token choice helps only when skipping is aggressive, and the  $L_2$  distance gives a small but steady gain over cosine because it keeps the size information the true error depends on. Since per-token choice is already optimal, the only thing that scales is proxy fidelity, set by the distance and, even more, by how rich the probe is. We suggest testing these methods on trained networks with fidelity-to-base-model scores, and improving the probe rather than the choice rule.

**Reproducibility.** All code and results run locally in pure NumPy: `latap_v2.py` (testbed), `run_core.py/agg_core.py` (12-seed core), `sweep.py/sweep_agg.py` (21-regime sweep), `probe_depth.py` (richness), `pressure_v2.py` (30-test suite), `plot_v2.py`; data in `results/`. See the repository README for exact commands.

## References

- [1] J. Ho, A. Jain, P. Abbeel. Denoising Diffusion Probabilistic Models. *NeurIPS*, 2020.
- [2] J. Song, C. Meng, S. Ermon. Denoising Diffusion Implicit Models. *ICLR*, 2021.
- [3] Y. Song, S. Ermon. Generative Modeling by Estimating Gradients of the Data Distribution. *NeurIPS*, 2019.
- [4] Y. Song et al. Score-Based Generative Modeling through Stochastic Differential Equations. *ICLR*, 2021.
- [5] W. Peebles, S. Xie. Scalable Diffusion Models with Transformers. *ICCV*, 2023.
- [6] B. Efron. Tweedie’s Formula and Selection Bias. *JASA*, 106(496), 2011.
- [7] G. J. Székely, M. L. Rizzo. Energy Statistics: A Class of Statistics Based on Distances. *J. Stat. Plan. Inference*, 2013.
- [8] X. Ma, G. Fang, X. Wang. DeepCache: Accelerating Diffusion Models for Free. *CVPR*, 2024.
- [9] P. Selvaraju et al. FORA: Fast-Forward Caching in Diffusion Transformer Acceleration. arXiv:2407.01425, 2024.
- [10] F. Liu et al. Timestep Embedding Tells: It’s Time to Cache for Video Diffusion Models (TeaCache). arXiv:2411.19108, 2024.
- [11] C. Zou et al. Accelerating Diffusion Transformers with Token-wise Feature Caching (ToCa). arXiv:2410.05317, 2024.
- [12] DiCache: Learning a Shallow-Layer Online Probe for Adaptive Diffusion Caching. arXiv:2508.17356, 2025.
- [13] TaylorSeer: From Reusing to Forecasting—Accelerating Diffusion Transformers with Taylor Prediction. arXiv:2503.06923, 2025.
- [14] B. C. Challagundla. Probe Reliability is an Emergent Property of Trained Diffusion Models. Companion technical report, 2025.
- [15] Token-Adaptive Predictor for Training-Free Diffusion Acceleration. arXiv:2603.03792 (no public code; reimplemented here).